

# Lecture 7

## • Multithreading

### • 并发编程 (Concurrent Programming)

- 并发是指多个计算在同一时间发生。
- 在并发编程中，基本的执行单元有两种：
  - 进程 (Processes)
    - 执行一个程序会启动一个进程（即正在运行的活动程序）。
    - 操作系统会为不同的进程分配独立的内存空间。
  - 线程 (Threads)
    - 一个进程可以拥有多个线程（至少有一个线程）。
    - 同一进程内的线程共享该进程的内存和资源。

### • Java多线程 (Java Multithreading)

- 运行Java程序会启动JVM (Java虚拟机)。
- 操作系统会为一个JVM分配一个进程。
- 通常，每个Java程序对应一个JVM进程。
- 一个JVM进程可以包含多个线程：
  - 主线程 (Main thread)：负责运行main()方法。
  - 用户线程 (User threads)：由开发者定义，用于执行具体任务。
  - 系统线程 (System threads)：被JVM内部使用，例如垃圾回收 (GC)、即时编译 (JIT) 等。

### • 创建与启动线程 (Creating & Starting Threads)

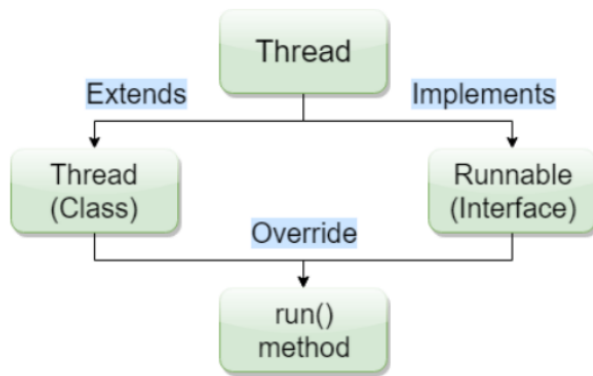
#### • Multithreading in Java (Java中的多线程)

- 当我们的Java程序启动时，主线程由JVM自动创建。
- 其他线程由主线程创建和管理。

```
public class Concurrency {  
    public static void main(String[] args){  
        System.out.println(Thread.currentThread().getName());  
    }  
}
```

Output "main"

- 创建与启动线程



- 方法1：继承Thread类（不推荐）
- 方法2：实现Runnable接口（推荐）

- **The Thread Class（Thread类）**

```
public class Cat extends Thread{
    @Override
    public void run(){
        System.out.printf("%s: cat\n", Thread.currentThread().getName());
    }

    public static void main(String[] args) {
        Cat cat = new Cat();
        cat.start();
        System.out.printf("%s: main\n", Thread.currentThread().getName());
    }
}
```

- 将一个类声明为Thread的子类。
- 这个子类需要重写Thread的run方法，指定该线程在run()中要做的事情。
- 然后可以创建这个子类的实例并启动线程。

-

```

public class CatThread extends Thread{
    int cnt = 0;

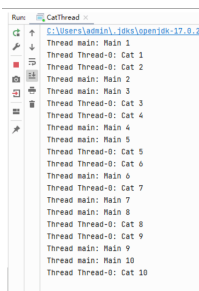
    @Override
    public void run(){
        while(cnt<10){
            System.out.printf("Thread %s: Cat %d\n",
                               Thread.currentThread().getName(), ++cnt);
            try{
                Thread.sleep(100);
            }catch (InterruptedException e){
                e.printStackTrace();
            }
        }
    }

    public static void main(String[] args) throws InterruptedException {

        Thread cat = new CatThread();
        cat.start();

        int cnt=0;
        while(cnt<10){
            System.out.printf("Thread %s: Main %d\n",
                               Thread.currentThread().getName(), ++cnt);
            Thread.sleep(100);
        }
    }
}

```



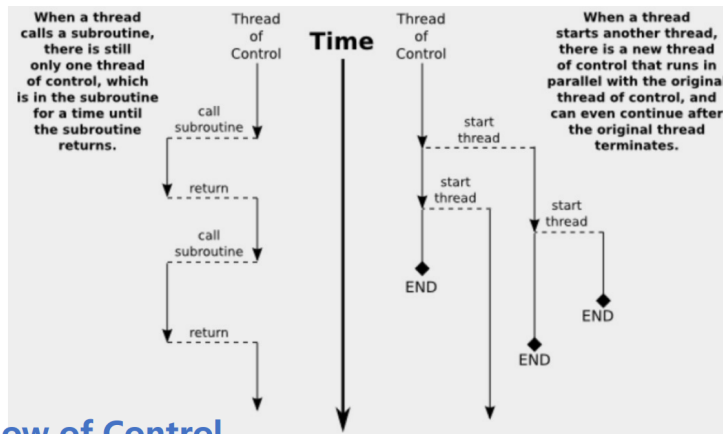
- Cat线程：
  - 打印线程名和计数，共打印10次。
  - 每次打印之间有一段时间间隔。
- 主线程（Main线程）：
  - 也打印线程名和计数，共打印10次。
  - 每次打印之间有一段时间间隔。

```

Thread cat = new CatThread();
//cat.start();
cat.run();

```

- run() 方法会像普通方法那样在当前线程中执行。
- start() 确实会创建一个新的线程，然后调用 run() 方法。
- start() 是非阻塞的：在执行后续操作时不需要等待它完成。
- 控制流（Flow of Control）



## Flow of Control

- （左侧）当线程调用一个子程序时，实际上只有一条控制流，在子程序执行期间，控制流位于子程序内，执行完成后才返回。
- （右侧）当线程启动另一个线程时，会有新的控制流与原有控制流并行运行，即使原来的线程结束，新的线程也可以继续运行。

### • Runnable 接口

- Runnable 接口应该被任何希望其实例能被线程执行的类实现（Thread 类本身也实现了该接口）。
- 为实现 Runnable，类必须实现抽象方法 run()。

```
@FunctionalInterface
public interface Runnable {
    When an object implementing interface Runnable is used to create a thread, starting the thread causes the object's run method to be called in that separately executing thread.
    The general contract of the method run is that it may take any action whatsoever.
    See Also: Thread.run()
    public abstract void run();
}
```

- 使用 Runnable 创建线程，而不是继承 Thread 类。
  - 线程有一个可以接收 Runnable 参数的构造函数
    - `Thread(Runnable target)`
    - 分配一个新的线程对象
  - 如果希望线程执行 run() 方法，可以将一个实现了 Runnable 接口的类、匿名类或 lambda 表达式的实例传递给 Thread 的构造函数

```
Runnable cat = () -> System.out.println("Cat thread");

Thread thread = new Thread(cat);
thread.start();
```

- 只需实现 Runnable 的 run() 方法，然后用它创建 Thread 对象并启动线程。代码中的 lambda 表达式实际上就是实现 Runnable 接口的方式，是一种简洁写法。
- Subclass vs Runnable

```

public static void main(String[] args) throws InterruptedException {
    // Cat thread (subclassing)
    Thread cat = new CatThread();
    cat.start();

    // Dog thread (Runnable)
    Runnable runnable = () -> {
        int cnt = 0;
        while(cnt < 10) {
            System.out.printf("Thread %s: Dog %d\n",
                               Thread.currentThread().getName(), ++cnt);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    };
    Thread dog = new Thread(runnable);
    dog.start();

    // Main thread
    int cnt=0;
    while(cnt<10){
        System.out.printf("Thread %s: Main %d\n",
                           Thread.currentThread().getName(), ++cnt);
        Thread.sleep(500);
    }
}

```

- 实际角度 (Practical POV)
  - Java 不支持多重继承
  - 如果一个类继承了 Thread, 它就不能再继承其他类
  - 如果一个类实现了 Runnable, 它仍然可以继承其他类
- 设计角度 (Design POV)
  - 在面向对象编程 (OOP) 中, 扩展一个类通常意味着添加新功能和修改/改进行为
  - 但是我们并没有真正改进线程的行为, 我们只是给它一个任务去运行 (task)
  - 实现 Runnable 能够将任务和它的执行进行解耦
- 同步机制 (Synchronization)
  - 同步机制确保同一时间只会有一个线程访问临界区 (共享资源)
  - 锁 (Lock)
    - Lock 用于控制那些要操作共享资源的线程
    - Lock 是一个接口, 无法直接创建 Lock 实例; 应该创建一个实现了 Lock 接口的类的实例
    - Java 提供了多个 Lock 的实现; ReentrantLock 是最常用的一个
    -

```

public class BankAccount {
    private double balance;
    private Lock balanceChangeLock;

    public BankAccount(){
        balance = 0;
        balanceChangeLock = new ReentrantLock();
    }

    public void withdraw(double amount){
        balanceChangeLock.lock();

        if(balance >= amount){
            balance -= amount;
            System.out.printf("Withdraw: %.1f, " +
                "new balance: %.1f\n", amount, balance);
        }

        balanceChangeLock.unlock();
    }

    public void deposit(double amount){
        balanceChangeLock.lock();

        balance += amount;
        System.out.printf("Deposit: %.1f, " +
            "new balance: %.1f\n", amount, balance);

        balanceChangeLock.unlock();
    }
}

```

- 当 Lock 实例被锁定时，其他线程调用 lock() 会被阻塞，直到锁定它的线程调用 unlock() 释放锁
- 当 unlock() 被调用时，Lock 会被解锁，其他线程可以再次获得锁
- Avoiding Deadlocks
  - Condition 接口 (java.util.concurrent.locks) 为线程提供了在给定条件为真之前挂起其执行的能力。
  - Condition 允许一个线程：
    - 暂时释放一个锁，以便另一个线程可以继续执行；
    - 当条件满足时，稍后重新获得该锁。

```

public class BankAccount {
    private double balance;
    private Lock balanceChangeLock;
    private Condition sufficientBalanceCondition;

    public BankAccount(){
        balance = 0;
        balanceChangeLock = new ReentrantLock();
        sufficientBalanceCondition = balanceChangeLock.newCondition();
    }
}

```

```

public void withdraw(double amount) throws InterruptedException {
    balanceChangeLock.lock();

    try{
        while(balance < amount){
            sufficientBalanceCondition.await();
        }
        balance -= amount;
        System.out.printf("Withdraw: %.1f, " +
            "new balance: %.1f\n", amount, balance);
    } finally {
        balanceChangeLock.unlock();
    }
}

public void deposit(double amount){
    balanceChangeLock.lock();

    try{
        balance += amount;
        System.out.printf("Deposit: %.1f, " +
            "new balance: %.1f\n", amount, balance);

        sufficientBalanceCondition.signalAll();

    } finally {
        balanceChangeLock.unlock();
    }
}

```

- 每个 Condition 对象属于一个特定的 Lock 对象。
- 一个 Lock 可以有一个或多个 Condition。
- 我们可以通过 Lock 接口的 newCondition() 方法获取一个 Condition 对象。
- 通常会给 Condition 对象起一个描述你要测试的条件的名称。
- 要使条件生效，我们需要实现一个适当的测试（即条件）。
- 只要条件不满足，就在 Condition 对象上调用 await() 方法（因此需要循环）。
- 在 Condition 对象上调用 await() 会使当前线程被挂起，并允许其他线程获取该锁对象。
- 当前线程会一直阻塞，直到某个其他线程在相同的 Condition 对象上调用 signalAll() 方法。
- 最终，它们中的某个线程将获得锁，并从 await() 方法中返回继续执行。

- synchronized

- 自 JDK 1.0 起, Java 中每个对象都有一个内在锁 (intrinsic lock / 内在锁)

```
public synchronized void method()
{
    method body
}

=

public void method()
{
    this.intrinsicLock.lock();
    try
    {
        method body
    }
    finally { this.intrinsicLock.unlock(); }
}
```

- 如果一个方法声明为 synchronized, 要调用该方法, 线程必须先获得拥有该方法的对象的内在锁
- 内在对象锁只有一个关联的条件 (single associated condition)
- 我们可以调用 wait() 来等待该条件, 并调用 notify()/notifyAll() 来解除等待线程的阻塞

```
public class BankAccount {
    private double balance;

    public BankAccount(){
        balance = 0;
    }

    public synchronized void withdraw(double amount)
        throws InterruptedException {

        while(balance < amount){
            wait();
        }
        balance -= amount;
        System.out.printf("Withdraw: %.1f, " +
            "new balance: %.1f\n", amount, balance);
    }

    public synchronized void deposit(double amount){
        balance += amount;
        System.out.printf("Deposit: %.1f, " +
            "new balance: %.1f\n", amount, balance);

        notifyAll();
    }
}
```

- 一个被 synchronized 修饰的实例方法, 是在拥有该方法的实例 (对象) 上进行同步的。
- 当一个线程正在为某个对象执行一个 synchronized 方法时, 所有其他调用同一对象上任何 synchronized 方法的线程都会被阻塞, 直到第一个线程对该对象的操作完成为止。



```

public class BankAccount {
    private double balance;

    public BankAccount() {
        balance = 0;
    }

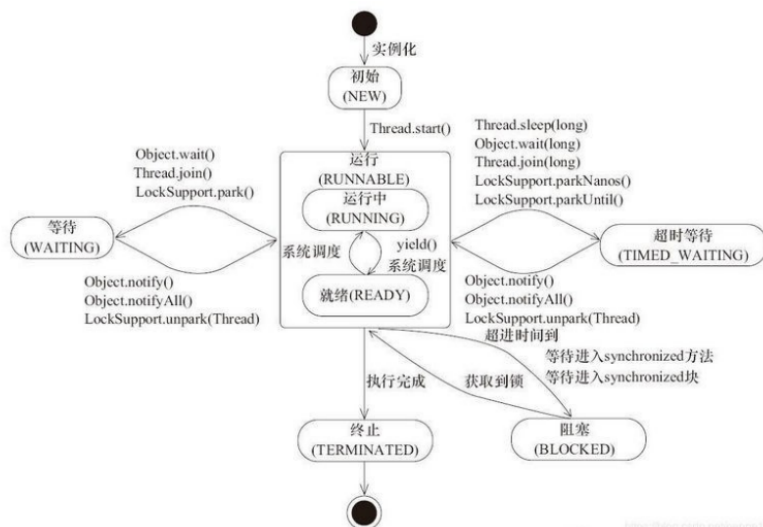
    public void withdraw(double amount)
        throws InterruptedException {
        synchronized (this) {
            while (balance < amount) {
                wait();
            }
            balance -= amount;
            System.out.printf("Withdraw: %.1f, " +
                "new balance: %.1f\n", amount, balance);
        }
    }

    public void deposit(double amount) {
        synchronized (this) {
            balance += amount;
            System.out.printf("Deposit: %.1f, " +
                "new balance: %.1f\n", amount, balance);

            notifyAll();
        }
    }
}

```

- synchronized 代码块只对方法的一部分进行同步。
  - 一个 synchronized 代码块在括号中接收一个对象，这个对象称为监视器对象 (monitor object) 。
  - 只有一个线程可以在对同一监视器对象同步的 Java 代码块内部执行。
- 线程状态 (Thread States)



- 一个线程可以处于下列状态之一（枚举 Thread.State）：
  - **NEW (新建)**
    - 当你使用 new 创建线程（例如 new Thread(r)）时，线程进入初始的 NEW 状态。
    - 在该状态下，程序尚未开始执行线程的代码。
  - **RUNNABLE (可运行/就绪或运行中)**
    - 已准备运行（调用 Thread.start() 后进入此状态）。

- 除了是否有可用的 CPU 可供执行之外，没有其他因素阻止该线程“运行”。
- **BLOCKED（阻塞，等待获取监视器/锁）**
  - 当线程尝试获取某个对象的内在锁（例如通过 synchronized）而该锁当前被其它线程持有时，线程将变为 BLOCKED（阻塞）状态。
  - 当所有其它线程释放该锁后，阻塞的线程会被解除阻塞并有机会获取该锁继续执行。
- **WAITING（等待，等待另一个线程显式唤醒）**
  - 在 WAITING 状态时，线程在等待来自另一个线程的信号。
  - 这通常通过调用 Object.wait() 或 Thread.join() 等方法实现。
  - 线程将保持在此状态，直到另一个线程调用 Object.notify()、Object.notifyAll()（或相应的解阻方法）唤醒它，或该线程终止。
- **TIMED\_WAITING（限时等待，例如 sleep、join(long)、wait(long) 等）**
  - 多种方法支持超时等待。
  - 调用这些方法会使线程进入 TIMED\_WAITING（限时等待）状态。
- **TERMINATED（终止）**
  - run() 方法正常返回时线程终止。
  - 如果 run() 方法因未捕获异常突然终止，线程也会进入终止状态。

## • 线程安全集合（Thread-safe Collections）

- Java 集合的并发性：
  - 所有 java.util 中的集合类（例如 ArrayList、HashMap、HashSet、TreeSet 等）默认都不是线程安全的（Vector 和 Hashtable 除外）。
  - 同步会带来开销。
    - Vector 和 Hashtable 是较早存在且从一开始就为线程安全设计的两个集合，但它们很快显现出较差的性能。
  - 新的集合（List、Set、Map 等）默认不提供并发控制，以在单线程应用中实现最大性能。
- 按线程安全机制可分为三类：
  - **写时复制（Copy-on-Write）集合**
    - 行为：顺序写入与并发读取
      - 读取不阻塞
      - 写操作不阻塞读操作，但同一时刻只允许一个写操作
    - 更适用于“读远多于写”的应用场景
    - 示例类：CopyOnWriteArrayList, CopyOnWriteArraySet
  - **基于比较并交换（CAS, Compare-and-Swap）的集合**
    - 先在本地图制变量的当前值（旧值）；
    - 计算新值；

- 调用 CAS（变量地址、旧值、新值）：
  - 检查变量是否仍等于旧值；
  - 如果是，则将变量设置为新值；
  - 否则重试（说明变量已被其他线程修改）。
- 示例类：ConcurrentLinkedQueue、ConcurrentSkipListMap
- 使用显式锁（Lock）的集合
  - 该机制将集合划分为可以分别加锁的部分，从而提高并发性。
  - 示例类：大多数 BlockingQueue 的实现，ConcurrentHashMap
    - BlockingQueue（阻塞队列）
      - 当发生【向已满的队列添加元素 / 从空队列移除元素】时，阻塞队列会导致线程阻塞：
      - 阻塞队列有助于协调多个线程的工作：
        - 生产者线程可以周期性地将中间结果放入阻塞队列；
        - 消费者线程可以从队列中移除中间结果并进一步处理。
      - 阻塞队列会自动平衡工作负载：
        - 如果生产者比消费者慢，消费者在等待结果时会被阻塞；
        - 如果生产者比消费者快，队列会阻塞直到消费者赶上为止。

```
BlockingQueue<Integer> queue = new LinkedBlockingQueue<>(5);
```

```
Runnable producer = ()->{
    try {
        queue.put(1);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
};
```

```
Runnable consumer = ()->{
    try {
        queue.take();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
};
```

- 如果尝试的操作不能立即完成，该方法调用将阻塞直到可以为止。
- 在内部，BlockingQueue 使用 ReentrantLock 和 Condition 来实现。

## • 任务与线程池（Tasks and Thread Pools）

- Thread Pools 线程池
  - 创建一个新线程代价高。
  - 如果程序会创建大量短生命周期的线程，不应把每个任务映射到一个独立线程，而应使用线程池。
  - 线程池包含若干已准备运行的线程。

- 你可以专注于实现任务逻辑，然后把任务提交给线程池执行。
- 将任务实现（业务逻辑）与任务执行解耦。
- 常用线程池：
  - `FixedThreadPool`：固定大小的线程池。如果提交的任务多于空闲线程，未被服务的任务将被放入队列，待其他任务完成后再运行。

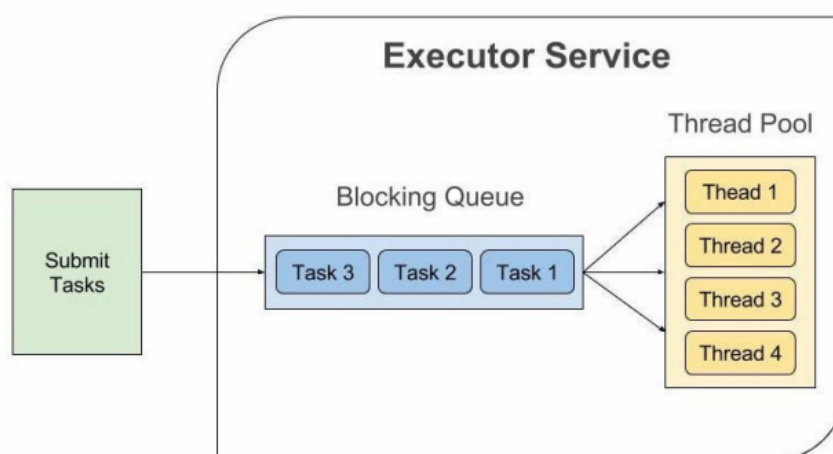
```
ExecutorService es1 = Executors.newFixedThreadPool( nThreads: 4);
for (int i = 0; i < 6; i++) {
    es1.submit(()-> System.out.println(Thread.currentThread().getName()
        + " processing the task"));
}
es1.shutdown();
```

```
pool-1-thread-2 processing the task
pool-1-thread-4 processing the task
pool-1-thread-1 processing the task
pool-1-thread-4 processing the task
pool-1-thread-2 processing the task
pool-1-thread-3 processing the task
```

- `SingleThreadExecutor`：退化为大小为1的线程池，单个线程按顺序逐个执行提交的任务。
  - `CachedThreadPool`：立即执行每个任务的线程池；若有空闲线程则复用，否则创建新线程。
- **Tasks 任务**
    - `Runnable` 是一种无参数、无返回值的异步方法接口；它封装了异步运行的任务。
    - `Callable` 与 `Runnable` 类似，但会返回一个值。
    - 例如，`Callable<Integer>` 表示一个最终返回 `Integer` 对象的异步计算。

```
public interface Callable<V>
{
    V call() throws Exception;
}
```

- **任务执行**



- `ExecutorService` 接口 (`java.util.concurrent.ExecutorService`) 表示一种异步执行机制，能够在后台并发执行任务。
- `ThreadPoolExecutor` 是 `ExecutorService` 的一个实现，提供一个线程池用于执行 `Runnable` 或 `Callable` 任务。

- Executors 类包含若干静态工厂方法，用于构造 ThreadPoolExecutor。
- 提交任务
  - 可以将 Runnable 或 Callable 提交给 ExecutorService 后会得到一个 Future 对象。
  - `Future` 保存异步计算的结果。
  - 常用方法签名：

- `Future<T> submit(Callable<T> task)`
- `Future<?> submit(Runnable task)`

- 使用线程池：

- 

```
ExecutorService executor =
    Executors.newFixedThreadPool(4);

// Define the task
Callable<String> task = .....;

// Submit the task
Future<String> future = executor.submit(task);

// Use Future to get the result asynchronously
String result = future.get(); // may block

executor.shutdown();
```

- 创建线程池：ExecutorService executor = Executors.newFixedThreadPool(4);
- 定义任务：Callable<String> task = ...;
- 提交任务并获得 Future：Future<String> future = executor.submit(task);
- 使用 Future 异步获取结果：String result = future.get(); (可能会阻塞)
- 不再提交任务时调用 executor.shutdown()。

- 

```
ExecutorService executor =
    Executors.newFixedThreadPool(4);

// Define the task
Runnable<String> task = .....;

// Execute the task
executor.execute(task);

executor.shutdown();
```

```
e:
di
•
•
```

- 定义任务：Runnable task = ...;
- 执行任务：executor.execute(task);

- execute 与 submit 的区别：
  - execute 不返回任何值（返回 void）；
  - submit 会返回一个 Future 对象，用于管理任务（例如取消、获取结果）。
- 控制一组任务：
  - 可以使用 ExecutorService 来控制一组相关任务：
  - 创建线程池、构造若干 Callable、调用 `invokeAny(callables)` 并打印返回的结果，然后关闭线程池

```
ExecutorService executorService =  
    Executors.newFixedThreadPool(3);  
  
Set<Callable<String>> callables = new HashSet<>();  
  
callables.add(() -> "Task 1");  
callables.add(() -> "Task 2");  
callables.add(() -> "Task 3");  
  
String result = executorService.invokeAny(callables);  
  
System.out.println("result = " + result);  
  
executorService.shutdown();
```

- invokeAny 方法将一组 Callable 对象（集合）中的所有对象提交执行，并返回某个已完成任务的结果。
- 你不知道哪个任务会先完成——通常是最先完成的那个。
- 在你愿意接受任意一个解的搜索问题中使用此方法
- 创建线程池、构造若干 Callable、调用 `invokeAll(callables)` 获取 futures 列表，遍历并使用 `future.get()` 获取每个任务的结果，最后关闭线程池

```

ExecutorService executorService = Executors.newFixedThreadPool(2);

List<Callable<String>> callables = new ArrayList<>();

callables.add() -> "Task 1";
callables.add() -> "Task 2";
callables.add() -> "Task 3";
callables.add() -> "Task 4";

List<Future<String>> futures = executorService.invokeAll(callables);

for(Future<String> future : futures){
    System.out.println("future.get = " + future.get());
}

executorService.shutdown();

```

- invokeAll 方法将一组 Callable 对象（集合）全部提交执行。
- 该方法会阻塞，直到所有任务都完成（即每个返回列表元素对应的 Future 的 isDone() 都为 true）。
- 方法返回表示所有任务结果的 Future 对象列表。
- 任务可能正常终止或异常终止，Future 会封装相应的异常信息。
- 常用方法：
  - `T invokeAny(Collection<? extends Callable<T>> tasks)`
  - `List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks)`